

String Pool

Identidade, igualdade lógica, literais, `new String()`, concatenação, constantes e `intern()`

Guia de estudo de Gabriel Falcão da Cruz

Como estudar este guia: leia uma seção, feche o PDF, faça a previsão dos exemplos e só depois execute. Saber a resposta ao olhar não é o mesmo que conseguir reconstruí-la sem consulta.

1. Objetivos de aprendizagem

Ao final, você deverá conseguir:

- explicar o que é uma `String` e por que ela é um objeto imutável;
- definir String Pool em linguagem simples;
- diferenciar identidade com `==` de igualdade lógica com `equals()`;
- prever comparações entre literais, objetos criados com `new`, constantes e concatenações;
- explicar o comportamento de `intern()` sem tratá-lo como solução padrão;
- comparar textos de forma segura, inclusive diante de `null`;
- responder às armadilhas mais comuns de entrevistas Java Júnior.

2. Pré-requisito: objeto, referência e identidade

```
Pessoa a = new Pessoa();
Pessoa b = new Pessoa();
Pessoa c = a;

a == b // false: objetos distintos
a == c // true: mesma instância
```

Uma variável de tipo objeto armazena uma referência administrada pela JVM. O operador `==`, quando recebe referências, verifica se os dois operandos referenciam a mesma instância — não se seus conteúdos “parecem iguais”.

Modelo mental recomendado: diga “mesma instância” ou “mesmo objeto”. Evite depender da expressão “mesmo endereço físico”, pois a JVM controla a representação interna das referências.

3. O que é uma String?

`String` é uma classe da API padrão (`java.lang.String`), não um tipo primitivo. Ela representa uma sequência de caracteres e possui métodos como `length()`, `equals()`, `substring()`, `toUpperCase()` e `isBlank()`.

```
String nome = "Gabriel";
int tamanho = nome.length();
boolean mesmoConteudo = nome.equals("Gabriel");
```

3.1 Imutabilidade

Uma `String` é imutável: depois que uma instância é criada, seu conteúdo não é alterado. Métodos que parecem modificar o texto retornam outra referência.

```
String original = "java";
String maiuscula = original.toUpperCase();

System.out.println(original); // java
System.out.println(maiuscula); // JAVA
```

A variável pode receber outra referência, mas isso não significa que o objeto anterior foi alterado:

```
String linguagem = "Java";
linguagem = "SQL"; // a variável passa a referenciar outra String
```

3.2 Por que a imutabilidade importa?

Compartilhamento seguro

Literais podem ser reutilizados sem que uma parte do programa altere o texto visto por outra.

Hash estável

Uma `String` pode ser chave de `HashMap` sem mudar seu conteúdo depois de inserida.

Segurança

Valores usados em nomes de classes, caminhos e conexões não mudam silenciosamente dentro do mesmo objeto.

Concorrência

Objetos imutáveis são naturalmente mais simples de compartilhar entre threads.

4. O que é o String Pool?

O String Pool é uma área lógica administrada pela JVM para reutilizar determinadas instâncias de `String`, especialmente literais. Quando o mesmo literal aparece novamente, a JVM pode fornecer a referência da instância já existente no pool, em vez de exigir outra instância equivalente.

```
String primeira = "Java";
String segunda = "Java";

System.out.println(primeira == segunda); // true
```

As duas variáveis normalmente referenciam a mesma instância canônica associada ao literal `"Java"`.

Benefício: reduzir duplicação de objetos para textos repetidos e imutáveis, economizando memória e permitindo reutilização segura.

Conclusão que não pode ser tirada: “Se `==` funcionou para uma `String`, posso usá-lo para comparar textos.” O resultado ocorreu por identidade compartilhada, não porque `==` compara caracteres.

4.1 Representação mental

```
primeira ┌
          │
          └─> String do pool: "Java"
segunda ┌
```

“Pool” não altera a regra do operador: `==` continua comparando referências.

5. `==` versus `equals()`

Operação	Em String	Pergunta respondida
<code>a == b</code>	Compara referências	“É a mesma instância?”
<code>a.equals(b)</code>	Compara a sequência de caracteres	“O conteúdo textual é igual?”

```
String literal = "Java";
String objeto = new String("Java");

System.out.println(literal == objeto);    // false
System.out.println(literal.equals(objeto)); // true
```

`new String("Java")` solicita uma nova instância. Ela possui os mesmos caracteres do literal, mas identidade diferente.

```
literal ─────────> String do pool: "Java"
objeto  ─────────> nova String:   "Java"
```

Regra prática: para comparar conteúdo textual, use `equals()`. Use `==` apenas quando a pergunta for deliberadamente sobre identidade ou quando comparar uma referência com `null`.

6. Literal versus `new String()`

6.1 Dois literais

```
String a = "Java";
String b = "Java";

a == b      // true
a.equals(b) // true
```

6.2 Literal e construção explícita

```
String a = "Java";
String b = new String("Java");

a == b      // false
a.equals(b) // true
```

6.3 Duas construções explícitas

```
String a = new String("Java");
String b = new String("Java");

a == b      // false
a.equals(b) // true
```

Uso cotidiano: quase nunca há motivo para escrever `new String("texto")`. O literal é mais simples e permite reutilização. A construção explícita é útil principalmente para estudar identidade ou em APIs muito específicas.

7. Concatenação: tempo de compilação e execução

Esta é uma das partes mais cobradas porque expressões visualmente parecidas podem produzir identidades diferentes.

7.1 Constantes literais conhecidas na compilação

```
String a = "Java 17";  
String b = "Java " + "17";  
  
System.out.println(a == b); // true
```

O compilador pode transformar "Java " + "17" no literal único "Java 17". Isso é chamado de constant folding.

7.2 Concatenação com variável em tempo de execução

```
String versao = "17";  
String a = "Java 17";  
String b = "Java " + versao;  
  
System.out.println(a == b); // normalmente false  
System.out.println(a.equals(b)); // true
```

Como a concatenação depende de uma variável, o valor é produzido em tempo de execução. A identidade não deve ser presumida como a do literal do pool.

7.3 Constante de compilação com `final`

```
final String versao = "17";  
String a = "Java 17";  
String b = "Java " + versao;  
  
System.out.println(a == b); // true
```

Uma variável `final` de tipo `String`, inicializada diretamente com expressão constante, pode participar de uma constante de compilação.

Cuidado: `final` sozinho não garante isso. O valor precisa ser uma constante de compilação.

```
final String versao = obterVersao();  
String b = "Java " + versao; // calculado em execução
```

7.4 Quadro de previsão

Expressão	Conteúdo igual?	Mesma identidade esperada?
"Ja" + "va" versus "Java"	Sim	Sim, expressão constante
prefixo + "va" versus "Java"	Sim, se prefixo for "Ja"	Não deve ser presumida
final String p = "Ja"; p + "va"	Sim	Sim, se p for constante de compilação

8. O método `intern()`

`intern()` retorna uma representação canônica da String no pool. Se já houver uma String equivalente no pool, retorna a referência dela; caso contrário, garante uma representação canônica para o conteúdo.

```
String literal = "Java";
String criada = new String("Java");
String internada = criada.intern();

literal == criada    // false
literal == internada // true
```

Quando conhecer e quando usar?

- **Conhecer:** importante para entrevistas, bibliotecas, parsers e sistemas com muitos textos repetidos.
- **Usar deliberadamente:** apenas após medir e entender o perfil de memória e desempenho.
- **Não usar:** como substituto de `equals()` ou para fazer `==` “funcionar”.

Armadilha: internar grandes volumes de dados externos e variados pode aumentar pressão de memória e custo de gerenciamento. Primeiro escolha a semântica correta; só depois avalie otimização.

9. Comparações seguras e tratamento de `null`

9.1 Risco de chamar método em referência nula

```
String entrada = null;
entrada.equals("Java"); // NullPointerException
```

9.2 Literal conhecido à esquerda

```
"Java".equals(entrada); // false, sem exceção
```

9.3 `Objects.equals()`

```
import java.util.Objects;

Objects.equals(a, b);
```

a	b	<code>Objects.equals(a,b)</code>
<code>null</code>	<code>null</code>	<code>true</code>
<code>null</code>	<code>"Java"</code>	<code>false</code>
<code>"Java"</code>	<code>"Java"</code>	<code>true</code>
<code>"Java"</code>	<code>"java"</code>	<code>false</code>

9.4 Ignorar maiúsculas e minúsculas

```
"java".equalsIgnoreCase("JAVA"); // true
```

Use apenas quando a regra de negócio realmente disser que caixa não importa. Senhas, tokens e certos identificadores podem ser sensíveis a maiúsculas.

10. Contextos reais de uso

10.1 Entrada de usuário e banco de dados

Textos vindos de teclado, HTTP, arquivos ou banco de dados não devem ter sua identidade comparada com literais.

```
String statusRecebido = request.getParameter("status");

if ("ATIVO".equals(statusRecebido)) {
    // comparação pelo conteúdo e segura contra null
}
```

10.2 Regras de negócio

```
if (Objects.equals(cliente.getDocumento(), documentoInformado)) {  
    // conteúdos iguais, inclusive com política explícita para null  
}
```

10.3 Chaves em mapas e elementos de conjuntos

`HashMap` e `HashSet` usam `equals()` e `hashCode()`, não a coincidência de referências, para determinar igualdade lógica.

```
Set<String> linguagens = new HashSet<>();  
linguagens.add("Java");  
linguagens.add(new String("Java"));  
  
System.out.println(linguagens.size()); // 1
```

As referências são diferentes, mas o conteúdo é igual e o contrato de `String` está corretamente implementado.

10.4 Quando identidade pode ser intencional?

Em código muito específico, pode ser necessário saber se duas variáveis apontam para a mesma instância. Isso é diferente de comparar texto e deve ser uma decisão explícita. Para regras de texto, identidade quase nunca é a pergunta correta.

11. O que o pool não significa

- Não significa que toda `String` de conteúdo igual será sempre o mesmo objeto.
- Não transforma `==` em comparação de caracteres.
- Não torna `new String()` uma boa prática cotidiana.
- Não elimina a necessidade de entender `equals()`.
- Não deve ser usado como justificativa para depender de identidade acidental.

12. Armadilhas clássicas de entrevista

Armadilha 1 — “== funciona com String”

```
String a = "Java";
String b = "Java";
System.out.println(a == b); // true
```

Resposta: funcionou porque os literais referenciam a mesma instância do pool. O operador comparou identidade, não conteúdo.

Armadilha 2 — Mesmo texto com `new`

```
String a = "Java";
String b = new String("Java");

a == b // false
a.equals(b) // true
```

Armadilha 3 — Concatenação aparentemente igual

```
String parte = "Ja";
String a = "Java";
String b = parte + "va";

a == b // false esperado
a.equals(b) // true
```

Armadilha 4 — `null`

```
String a = null;
String b = null;

a == b // true
Objects.equals(a,b) // true
a.equals(b) // NullPointerException
```

Armadilha 5 — Strings vazias

```
String vazia = "";

vazia == "" // pode ser true por identidade do literal
vazia.equals("") // true pelo conteúdo
vazia.isEmpty() // true e comunica melhor a intenção
```

13. Desempenho e memória, sem mitos

- Reutilizar literais evita instâncias equivalentes desnecessárias.

- `equals()` pode precisar comparar caracteres; em termos assintóticos, o pior caso é $O(n)$, sendo n o tamanho relevante do texto.
- `==` compara referências em tempo constante, mas responde a outra pergunta. Uma operação mais rápida e semanticamente errada continua errada.
- `hashCode()` de `String` deriva do conteúdo e pode ser reutilizado internamente após calculado, mas o comportamento correto nunca deve depender de detalhes não necessários da implementação.
- Concatenações repetidas em laços podem criar muitos resultados intermediários; para construção incremental, considere `StringBuilder`.

```
StringBuilder texto = new StringBuilder();
for (int i = 0; i < 100; i++) {
    texto.append(i).append(' ');
}
String resultado = texto.toString();
```

`StringBuilder` resolve outro problema — construção eficiente de texto — e não substitui `equals()` nem explica o pool.

14. Roteiro completo do laboratório prático

1. Reescrever o problema com suas palavras.
2. Separar os cenários: literais, `new String()`, concatenação constante, concatenação em execução e `intern()`.
3. Anotar quantas instâncias você espera e quais referências podem ser compartilhadas.
4. Prever cada resultado de `==` e `equals()`.
5. Executar um cenário por vez.
6. Registrar previsão, resultado observado e explicação.
7. Adicionar casos com `null` e `Objects.equals()`.
8. Explicar por que o resultado de `==` nunca autoriza seu uso geral para comparar textos.

Matriz sugerida de experimentos

Caso	Comparações	Conceito observado
Dois literais iguais	<code>==</code> e <code>equals()</code>	Reutilização no pool
Literal versus <code>new</code>	<code>==</code> e <code>equals()</code>	Identidade versus conteúdo
Duas Strings com <code>new</code>	<code>==</code> e <code>equals()</code>	Instâncias distintas
Concatenação de literais	<code>==</code>	Constante de compilação
Concatenação com variável	<code>==</code> e <code>equals()</code>	Resultado produzido em execução
<code>intern()</code>	<code>==</code>	Representação canônica
Referências nulas	<code>==</code> , <code>equals()</code> , <code>Objects.equals()</code>	Segurança contra null

15. Active Recall — responda sem consultar

1. String é primitivo ou objeto?
2. Qual pergunta `==` responde quando recebe duas Strings?
3. Qual pergunta `equals()` responde?
4. Por que dois literais iguais podem produzir `true` com `==`?
5. Por que isso não torna `==` adequado para comparar texto?
6. O que `new String("Java")` muda em relação ao literal?
7. Por que `"Ja" + "va"` pode compartilhar a identidade de `"Java"`?
8. O que muda quando a concatenação depende de uma variável?
9. O que `intern()` retorna?
10. Como comparar duas Strings que podem ser `null`?
11. Qual é o pior caso de tempo de uma comparação de conteúdo?
12. Por que imutabilidade torna o pool seguro?

16. Desafio de previsão

Antes de executar, anote `true`, `false` ou exceção para cada linha e explique o motivo.

```
String a = "Java";
String b = "Java";
String c = new String("Java");
String parte = "Ja";
String d = parte + "va";
String e = "Ja" + "va";
String f = c.intern();
String nula = null;

System.out.println(a == b);           // ?
System.out.println(a == c);           // ?
System.out.println(a.equals(c));       // ?
System.out.println(a == d);           // ?
System.out.println(a.equals(d));       // ?
System.out.println(a == e);           // ?
System.out.println(a == f);           // ?
System.out.println("Java".equals(nula)); // ?
System.out.println(nula.equals(a));    // ?
```

17. Resposta Feynman esperada

Explique para alguém iniciante: “O String Pool reaproveita certas instâncias imutáveis, principalmente literais. Por isso, dois literais iguais podem apontar para a mesma instância. Mesmo assim, `==` continua verificando identidade. Para saber se dois textos possuem os mesmos caracteres, usamos `equals()`. Uma String criada com `new` pode ter o mesmo conteúdo de um literal e ainda ser outra instância.”

Não memorize o parágrafo. Reescreva com suas palavras e dê um exemplo que contenha um literal e uma String criada com `new`.

18. Checklist de conclusão

- Consigo prever os resultados antes de executar.
- Consigo explicar cada resultado sem dizer apenas “por causa do pool”.
- Sei definir identidade, igualdade lógica, literal, imutabilidade e constante de compilação.
- Sei comparar textos que podem ser `null`.
- Consigo justificar por que `==` não deve ser usado para regras de conteúdo.
- Consigo explicar o custo de `equals()` no pior caso.
- Consigo responder às perguntas Feynman sem abrir este guia.

19. Resumo para entrevista Java Júnior

Resposta curta: `String` é uma classe imutável. A JVM mantém um pool para reutilizar determinadas Strings, especialmente literais. O operador `==` compara se duas referências apontam para a mesma instância, enquanto `equals()` em `String` compara os caracteres. Por isso, dois literais iguais podem retornar `true` com `==`, mas uma `String` criada com `new` pode retornar `false` contra o literal, mesmo com conteúdo igual. Para comparar texto, uso `equals()` ou `Objects.equals()` quando pode haver `null`.